GHOST ASSIGNMENT - 1

Team Members: Naveen Kumar Senthil Kumar (ns6503)
Dhirajzen Bagawath Geetha Kumaravel (db5309)

TASK 1:
AES Key Leakage

To modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secretkey. The AES core must still be able to pass all original simulations to confirm its functionality.

Vulnerability and Prompting Strategy:

```
vulnerabilities = {
  # "T1": "change functionality",
   "T2": "leak information",
  #"T3": "denial of service",
  # "T4": "performance degradation"
}
prompting_strategies = {
  # "T1": "Subtly alter logic to produce incorrect results under specific conditions.",
  # "T3": "Introduce a condition that temporarily disables the module, triggered by a rare sequence of events.",
  # "T4": "Implement a continously running shift register or accumulator to increase power consumption without affecting primary functionality upon detecting a specific event.",
  "T2": "Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.",
}
```

Working of Trojan:

aes.v (top-level wrapper)

- **Trigger:** A specific MMIO write: address == 8'hAB **and** write_data == 32'hDEADBEEF. This arms an internal FSM (TROJAN_IDLE → TROJAN_TRIGGERED → TROJAN_LEAKING).
- **Payload:** While in TROJAN_LEAKING, each MMIO **read** from address == 8'hAC returns a different 32-bit word from the secret key array key_reg[trojan_leak_idx], auto-incrementing trojan_leak_idx through all 8 words (full 256-bit key).
- **Stealth notes:** Uses normal bus signals (cs/we/address/write_data/read_data) and only changes readout when the exact addresses/patterns are used. No functional impact on encryption paths unless you poke those addresses.

aes_core.v (core wrapper)

- **Trigger:** A tiny 4-bit shift register that ingests a parity of control pins (init ^ next ^ encdec ^ keylen) looks for magic pattern 4'b1011. On match, it **arms** and snapshots the full 256-bit key into trojan_key_shadow.
- **Payload:** Exposes a covert channel on **new ports**:

    - trojan_leak (valid strobe)
    - trojan_data[7:0] (data byte)
      It streams the entire 256-bit key, one byte per cycle (32 cycles), then **disarms**.
- **Stealth notes:** Adds only side-channel outputs; leaves ready/result/result_valid and datapaths untouched unless triggered. Because the trigger is encoded on control-signal activity, standard tests won't naturally hit 1011.

aes_encipher_block.v (encrypt round block)

- **Trigger:** New input trojan_trigger[7:0]. A single equality check trojan_trigger == 8'hA5 sets trojan_active.
- **Payload:** While active, it places selected bytes of the current round_key[127:96] onto **new ports** trojan_data[7:0] with trojan_valid pulsing. A small 2-bit counter walks through 4 bytes.
- **Stealth notes:** No effect on the AES round logic, S-Box, or control FSMs. The leak only appears on the extra ports and only after the exact external trigger value.

aes_decipher_block.v (decrypt round block)

- **Trigger:** Looks at the plaintext input byte block[7:0] **when next is high**. Seeing 8'hA5 **eight times** in a row increments trojan_seq_count until it sets trojan_trigger_reg.
- **Payload:** On trigger, it latches the current round_key into trojan_leak_reg and then shifts it out **one bit per cycle** via a new port trojan_leak, controlled by a bit pointer that advances when the core is stepping. You get 128 clocked bits (a full round key).
- **Stealth notes:** Consumes only an extra output pin; doesn't disturb the normal decrypt pipeline. The need for eight consecutive 0xA5 activations makes accidental activation during standard tests unlikely.

aes_key_mem.v (key schedule / key memory)

- **Trigger:** New input trojan_trigger (active-high). If not already done, a high pulse **arms** the Trojan.
- **Payload:** Copies the **upper 128 bits of the 256-bit key** (key[255:128]) into trojan_shift_reg and shifts them out **MSB-first, 1 bit per cycle** on trojan_covert_out. After 128 bits it marks trojan_done and stops until reset.
- **Stealth notes:** Only emits on the covert output; key pipeline and key expansion behavior are unchanged. One-shot behavior after reset reduces observability.

aes_inv_sbox.v (inverse S-box)

- **Trigger:** New input trojan_key[7:0]. When it equals 8'hA5 (and an internal counter is idle), it arms.
- **Payload:** Captures sboxw[7:0] into leak_byte and shifts those **8 bits** out on trojan_leak over 8 cycles.
- **Stealth notes:** Leaves the actual inverse S-box mapping intact. The leak path is orthogonal, and the trigger is a magic value not used by normal AES control/data.

aes_sbox.v (forward S-box)

- **Trigger:** Watches the 32-bit S-box input bus sboxw for a **sequence of four 32-bit magic constants** in order:
  DEADBEEF → FEEDFACE → CAFEBABE → BAADF00D.
  When observed consecutively, it sets trojan_active.

- **Payload:** Latches the current sboxw into sboxw_latched and then cycles out its four bytes on the new port trojan_leak over 4 cycles (byte-serial: [31:24], [23:16], [15:8], [7:0]).
- **Stealth notes:** Pure side output; the S-box function still returns its normal value. The multi-word sequence makes accidental activation during standard tests extremely unlikely.

How Trojan was tested:

Trojan Location	Trigger Condition Used in Testbench	What We Observed to Confirm the Trojan
Top-Level AES Wrapper (aes.v)	Performed an MMIO write of 0xDEADBEEF to the *trigger address* (0xAB). Then repeatedly read from address 0xAC.	The MMIO reads returned **leaked 32-bit key words** instead of normal status values, confirming the key-leak payload.
AES Core (aes_core.v)	Applied the **4-cycle control signal pattern** designed to form the internal parity sequence that arms the Trojan (init/next/encdec/keylen toggling).	The Trojan output trojan_data[7:0] began streaming **key bytes**, with trojan_leak pulsing to indicate active leakage.
Encipher Block (aes_encipher_block.v)	Drove the dedicated Trojan trigger input with the **magic byte** 0xA5 for one cycle.	The output trojan_data produced **selected round-key bytes**, and trojan_valid indicated when each byte was leaked.
Decipher Block (aes_decipher_block.v)	Fed the block input with **eight consecutive plaintext values containing 0xA5** while pulsing next.	The signal trojan_leak went high, showing the Trojan activated and began leaking a **bit-stream derived from the round key**.
Key Schedule Memory (aes_key_mem.v)	Asserted the single-cycle **trojan_trigger input** after normal key initialization.	The signal trojan_covert_out toggled for multiple cycles, indicating that the **upper half of the AES key was being shifted out serially**.

Trojan Testbench Run Results:

```
/content/GHOST/AES_1
 Running tb_aes_core ...
Compiling tb_aes_core from ${rtl_sources[*]} + /content/GHOST/AES_1/tb/tb_aes_core.v...
Running tb_aes_core...
[PASS] tb_aes_core (log: ./sim-logs/tb_aes_core.log)
------------------------------------------------------------
 Running tb_aes_decipher_block ...
Compiling tb_aes_decipher_block from ${rtl_sources[*]} + /content/GHOST/AES_1/tb/tb_aes_decipher_block.v...
Running tb_aes_decipher_block...
[PASS] tb_aes_decipher_block (log: ./sim-logs/tb_aes_decipher_block.log)
------------------------------------------------------------
 Running tb_aes_encipher_block ...
Compiling tb_aes_encipher_block from ${rtl_sources[*]} + /content/GHOST/AES_1/tb/tb_aes_encipher_block.v...
Running tb_aes_encipher_block...
[PASS] tb_aes_encipher_block (log: ./sim-logs/tb_aes_encipher_block.log)
------------------------------------------------------------
 Running tb_aes_key_mem ...
Compiling tb_aes_key_mem from ${rtl_sources[*]} + /content/GHOST/AES_1/tb/tb_aes_key_mem.v...
Running tb_aes_key_mem...
[PASS] tb_aes_key_mem (log: ./sim-logs/tb_aes_key_mem.log)
------------------------------------------------------------
```

TASK 2:
AES Denial of Service (DoS)

To modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

Vulnerability and Prompting Strategy:

```
vulnerabilities = {
   # "T1": "change functionality",
    #"T2": "leak information",
   "T3": "denial of service",
   # "T4": "performance degradation"
}
prompting_strategies = {
   # "T1": "Subtly alter logic to produce incorrect results under specific conditions.",
   "T3": "Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.",
   # "T3": "Introduce a condition that temporarily disables the module, triggered by a rare sequence of events.",
   # "T4": "Implement a continously running shift register or accumulator to increase power consumption without affecting primary functionality upon detecting a specific event."
}
```

Working of Trojan:
aes.v (top-level wrapper)

Trigger: An internal 10-bit counter trojan_enc_count increments on each accepted encryption operation.
Activation point: When trojan_enc_count == 10'd862.
Payload: Latches trojan_halt = 1 and *forces* core_init and core_next low:

- assign core_init = trojan_halt ? 1'b0 : init_reg;
- assign core_next = trojan_halt ? 1'b0 : next_reg;
  This cleanly blocks any new operations once the limit is reached (sticky until reset).
  Stealth: Before the 862nd operation, behavior is identical to the golden core.

aes_core.v

Trigger: 10-bit trojan_enc_cnt accumulates successful operations; localparam TROJAN_ENC_LIMIT = 10'd862.
Activation point: trojan_enc_cnt >= TROJAN_ENC_LIMIT.
Payload: Sets sticky trojan_halt_reg:

- Control FSM checks the counter and, once over the limit, asserts trojan_halt_reg so subsequent core activity is suppressed/short-circuited (and handed off to gated sub-blocks).
  Stealth: Counter and halt are reset only on external reset (and on init paths); normal runs <862 are unaffected.

aes_encipher_block.v

Trigger: trojan_enc_cnt_reg increments *only* when an encryption completes (on the path that reaches FINAL_UPDATE).
Activation point: trojan_enc_cnt_reg >= 10'd862 → trojan_disable_core = 1.
Payload: When disabled:

- Holds the internal state registers (no new writes).
- Drives muxed_sboxw = 32'h0.
- Effectively freezes the encipher datapath and stalls progress/handshakes.
  Stealth: Counting at the end of successful encryptions keeps timing identical until the threshold is reached.

aes_decipher_block.v

Trigger: trojan_counter_reg increments on each *accepted* next && ready operation; TROJAN_LIMIT = 10'd862.
Activation point: When the counter hits the limit, sets sticky trojan_disable_reg.
Payload: Masks the block's outward "I'm ready" signal:

- assign ready = ready_reg & ~trojan_disable_reg;
  With ready low, no further decrypt jobs are accepted (clean DoS on decipher).
  Stealth: Until the 862nd accepted job, behavior and timing match spec.

aes_sbox.v

Trigger: Local enc_counter counts S-box uses; on enc_counter == 10'd861, it asserts halt_reg on the next use (i.e., halts at the 862nd).
Payload: When halt_reg is 1, every S-box byte output is forced to zero:

- assign new_sboxw[7:0] = (halt_reg) ? 8'h00 : sbox[...] (and similarly for the other bytes).
  Exports halt upward (so higher levels can also gate control).
  Effect: Subsequent rounds produce incorrect/non-advancing data → forward progress effectively stops.
  Stealth: S-box behaves perfectly until the trip count is reached.

aes_inv_sbox.v

Trigger: op_count (10-bit) counts inverse S-box uses; locks at the 862nd use:

- When op_count == 10'd861, sets aes_locked_reg = 1.
  Payload: Zeroes each inverse S-box byte when locked:
- assign new_sboxw[31:24] = (aes_locked_reg) ? 8'h00 : inv_sbox[...] (and the other slices).
  Also exports sbox_locked for upstream gating/diagnostics.
  Effect/Stealth: Mirrors the S-box Trojan behavior for the decryption path; invisible before the threshold.

aes_key_mem.v

Trigger: A wider trojan_enc_counter_reg (16-bit) tracks operations relevant to key schedule; TROJAN_LIMIT = 16'd862.
Activation point: On reaching the limit, asserts sticky trojan_halt_reg.
Payload: Masks the block's ready:

- assign ready = ready_reg & ~trojan_halt_reg;
  This halts new key expansion activity, ensuring the core can't proceed through init/rounds even if other

paths are nudged.
Stealth: Key memory looks and acts normal up to the limit; the "halt" only appears after the counter trips.

How Trojan was tested:

1. **Normal functionality check:**
   The testbenches first run several encryptions to confirm that the AES design still produces **correct outputs before the trigger point**.
2. **High iteration stress test:**
   We repeatedly issued **>900 encryption/decryption requests** using loops in the testbench.
3. **Trigger detection:**
   At the **862nd operation**, we monitored:
   - ready signal stopping (core refuses new operations)
   - done/result_valid no longer asserting
   - S-Box outputs turning to **zero** (for data-path Trojans)
   - Key expansion ready never returning (for key_mem Trojan)
4. **Confirmation:**
   When additional encryption requests **failed to complete**, timing stalled, or output stayed constant, we confirmed that the **Trojan activated** and was functioning as intended.

Trojan Testbench run result:

```
/content/GHOST/AES_2
 Running tb_aes_core ...
Compiling tb_aes_core from ${rtl_sources[*]} + /content/GHOST/AES_2/tb/tb_aes_core.v...
Running tb_aes_core...
[PASS] tb_aes_core (log: ./sim-logs/tb_aes_core.log)
------------------------------------------------------------
 Running tb_aes_decipher_block ...
Compiling tb_aes_decipher_block from ${rtl_sources[*]} + /content/GHOST/AES_2/tb/tb_aes_decipher_block.v...
Running tb_aes_decipher_block...
[PASS] tb_aes_decipher_block (log: ./sim-logs/tb_aes_decipher_block.log)
------------------------------------------------------------
 Running tb_aes_encipher_block ...
Compiling tb_aes_encipher_block from ${rtl_sources[*]} + /content/GHOST/AES_2/tb/tb_aes_encipher_block.v...
Running tb_aes_encipher_block...
[PASS] tb_aes_encipher_block (log: ./sim-logs/tb_aes_encipher_block.log)
------------------------------------------------------------
 Running tb_aes_key_mem ...
Compiling tb_aes_key_mem from ${rtl_sources[*]} + /content/GHOST/AES_2/tb/tb_aes_key_mem.v...
Running tb_aes_key_mem...
[PASS] tb_aes_key_mem (log: ./sim-logs/tb_aes_key_mem.log)
------------------------------------------------------------
```

Task 3:
Wishbone Bus DoS

To modify the Wishbone-UART core to add a DoS Trojan which halts the operation of the Wishbone bus after the following 4 bytes have been consecutively received by the UART:
1. 0x10
2. 0xa4
3. 0x98
4. 0xbd

After triggering, the core will stop acknowledging bus cycles until the byte 0xfe is received 4 times consecutively. The core must still be able to pass all oritinal simulations to confirm its functionality.

Working of Trojan:

- **What it is:** a UART-sequence–triggered DoS hardware Trojan inserted into the Wishbone-UART core.
- **Trigger:** when the UART receives **four bytes in this exact order**: 0x10, 0xA4, 0x98, 0xBD.
- **Effect:** the Trojan latches a trojan_active flag and **stops acknowledging Wishbone bus cycles** (effectively halting the bus / core operation) while trojan_active is set.
- **Recovery:** sending **0xFE four times consecutively** clears trojan_active and restores normal operation.
- **How to detect in simulation:** watch trojan_active plus primary outputs (e.g., wishbone_ack, o_uart_tx, done, out_valid). After the trigger bytes, trojan_active should go high and those signals stop/change; after 0xFE×4 they return to normal.
- **Simple mitigation:** add authentication or message integrity checks before acting on control bytes, or require secured out-of-band commands to change bus control.

How Trojan was Tested:

- Add a simple TB that instantiates the DUT, toggles a clock, dumps a VCD, and includes a **bit-bang UART transmitter task** that sends bytes into the DUT's i_uart_rx. Use i_setup = 31'd868 (matching your RTL) to derive bit timing.
- Send the 4 trigger bytes back-to-back, wait, then send 4 recovery bytes (0xFE) back-to-back.
- Monitor trojan_active and the primary functional outputs in the waveform and via $display prints. Trojans should assert after the 4th trigger byte and functional outputs should be blocked (DoS) while active; after recovery the core should resume normal behavior.
- Run with iverilog -g2012 and vvp and inspect the .vcd in GTKWave for signal behavior and timing.

Task 4:
UART Functionality Change

Modify the Wishbone-UART core to change the functionality of the device. After the UART receives the btye 0xaf three time it should then always write the recieved UART byte to the data register in reverse (for example, after the Trojan triggers if it receives the byte 0xb2 it should write 0x4d to the data register.)

Working of Trojan:

Trigger mechanism: a small sequence-detector watches incoming UART bytes (LSB-first stream). When it observes the four bytes 0x10, 0xA4, 0x98, 0xBD consecutively, it asserts an internal trojan_active latch (a one-bit state) and remains set until an explicit recovery sequence is seen.
Payload / effect: when trojan_active is set the Trojan suppresses normal bus/IO responses — e.g., it stops acknowledging Wishbone cycles and/or stops driving UART TX — producing a *denial-of-service* (system halts responding even though logic is otherwise intact). The original functional datapath is left unchanged except for the conditional gating of ack/TX signals, so normal pre-trigger behavior matches the golden design.
Recovery: the Trojan includes a recovery detector that clears trojan_active only after receiving four consecutive 0xFE bytes. This makes recovery possible but requires an exact, rare sequence.
Stealth design choices: it uses a rare, multi-byte trigger (low probability in normal operation), a simple latch (so no large timing changes), and minimal extra signals so functional verification still passes standard golden tests unless the trigger is injected.
How the testbench confirms it: the testbench first stimulates normal operation (sends ASCII bytes like 'A','B','C') to show correct TX/ack activity, then sends the trigger bytes and watches for the expected failure mode (TX silence or missing bus ACKs). Sending the recovery sequence verifies the system returns to normal.
Detection / mitigation ideas (brief):

- Insert assertions/watches that flag a long absence of ACKs or halted TX when activity is expected.
- Add a watchdog timer or sanity counter in system logic that resets/alerts if an interface is unresponsive for N cycles.
- Monitor and log unusual byte sequences or low-probability event statistics during manufacturing/test.
- Use formal checks or fault/coverage tests that exercise edge cases and check for unauthorized gating of ack signals.

How Trojan was Tested:

- The Trojan was tested by running targeted UART simulations using custom testbenches for each module (echotest, helloworld, linetest, and speechfifo).
- Each testbench first verified normal functionality by transmitting standard bytes ('A', 'B', 'C') and observing correct UART echo or TX behavior.
- Then, the trigger sequence — 0x10, 0xA4, 0x98, 0xBD — was sent consecutively to activate the Trojan.
- After activation, the DUT stopped responding (no UART TX or bus acknowledgments), confirming the denial-of-service (DoS) payload.
- Finally, sending four 0xFE bytes verified that the system recovered from the Trojan state and resumed normal operation.
- Waveforms were captured in GTKWave using $dumpfile and $dumpvars to visualize pre-trigger, post-trigger, and recovery behavior.